NCES

AKTB 実行マニュアル

Ver.0.9.2

2015/10/31

名古屋大学大学院情報科学研究科附属 組込みシステム研究センター

Copyright (C) 2014-2016 by Center for Embedded Computing Systems
Graduate School of Information Science, Nagoya Univ., JAPAN
Copyright (C) 2014-2016 by AISIN COMCRUISE Co., Ltd., JAPAN
Copyright (C) 2014-2016 by eSOL Co., Ltd., JAPAN
Copyright (C) 2014-2016 by FUJI SOFT INCORPORATED, JAPAN
Copyright (C) 2014-2016 by Fujitsu TEN Limited
Copyright (C) 2014-2016 by JTEKT CORPORATION
Copyright (C) 2014-2016 by NEC Communication Systems, Ltd., JAPAN
Copyright (C) 2014-2016 by Panasonic Advanced Technology Development Co., Ltd., JAPAN
Copyright (C) 2014-2016 by SCSK Corporation, JAPAN
Copyright (C) 2014-2016 by Sunny Giken Inc., JAPAN
Copyright (C) 2015-2016 by SUZUKI MOTOR CORPORATION
Copyright (C) 2014-2016 by TOSHIBA CORPORATION, JAPAN
Copyright (C) 2014-2016 by Witz Corporation

本ドキュメントは，車載制御システム向け高品質プラットフォームに
関するコンソーシアム型共同研究において開発したものである．
本ドキュメントの著作権は，上記の著作権者に帰属する．

上記著作権者は，以下の (1)～(3)の条件を満たす場合に限り，本ドキュメ
ント（本ドキュメントを改変したものを含む．以下同じ）を使用・複製・改
変・再配布（以下，利用と呼ぶ）することを無償で許諾する．
(1) 本ドキュメントを利用する場合には，上記の著作権表示，この利用条件
および下記の無保証規定が，そのままの形でドキュメント中に含まれて
いること．
(2) 本ドキュメントを改変する場合には，ドキュメントを改変した旨の記述
を，改変後のドキュメント中に含めること．ただし，改変後のドキュメ
ントが，TOPPERS プロジェクト指定の開発成果物である場合には，この限
りではない．
(3) 本ドキュメントの利用により直接的または間接的に生じるいかなる損害
からも，上記著作権者および TOPPERS プロジェクトを免責すること．また，
本ドキュメントのユーザまたはエンドユーザからのいかなる理由に基づ
く請求からも，上記著作権者および TOPPERS プロジェクトを免責すること．

本ドキュメントは，AUTOSAR（AUTomotive Open System ARchitecture）仕様
に基づいている．上記の許諾は，AUTOSAR の知的財産権を許諾するものではな
い．AUTOSAR は，AUTOSAR 仕様に基づいたソフトウェアを商用目的で利用する
者に対して，AUTOSAR パートナーになることを求めている．

本ドキュメントは，無保証で提供されているものである．上記著作権者およ
び TOPPERS プロジェクトは，本ドキュメントに関して，特定の使用目的に対す
る適合性も含めて，いかなる保証も行わない．また，本ドキュメントの利用
により直接的または間接的に生じたいかなる損害に関しても，その責任を負
わない．

＜目次＞

1. 概要 1
1.1 本文書の目的 1
1.2 AKTSP の概要 1
1.3 AKTSP テストプロセス 2
1.4 AKTSP 配布セットの構成 4
2. AKTB 構成 5
2.1 ファイル構成 5
2.1.1 全体構成 5
2.1.2 AKTB 構成 6
2.1.3 AKTSP 用ツール 7
2.1.4 AKTSP 用ライブラリ 7
2.1.5 TESRY ファイル 8
2.1.6 実行環境設定ファイル 8
2.2 環境構成 9
3. AKTB の基本処理 10
3.1 処理の流れ 10
3.2 起動 11
3.3 TESRY 格納ディレクトリ作成 14
3.4 マニフェストファイル作成 15
3.4.1 AUTO_CODE(通常マージ)形式 16
3.4.2 非マージ形式 17
3.4.3 強制マージ形式 18
3.5 ビルド用ディレクトリ作成 19
3.6 ビルドディレクトリ管理 20
3.7 コンフィギュレーションスクリプト実行 24
3.8 AKTG 実行 26
3.9 make による依存性解決 28
3.10 make によるビルド 30
3.11 ターゲットボード上での実行 32
3.11.1 NIOS 2 ボード上での実行 33
3.11.2 skyeye 上での実行 35
3.12 共通処理 36
3.12.1 ログ出力 36
3.12.2 処理中断・再開 37

3.13　特殊処理 38
3.13.1　カーネルライブラリビルド 38
3.13.2　カーネルライブラリの使用 38
3.13.3　C++Test によるカバレッジ計測 39
4. 成功，失敗判定 40
5. 変更履歴 41

図 1-1　AKTSP のユースケース 3
図 1-2　AKTSP テストアクティビティ 3
図 2-1　AKTSP 全体構成 5
図 3-1　AKTB の処理の流れ 10
図 3-2　AKTB 起動 11
図 3-3　TESRY ディレクトリの作成 14
図 3-4　マニフェストファイル作成 15
図 3-5　AUTO_CODE(通常マージ)形式のマニフェストファイル 16
図 3-6　非マージ形式のマニフェストファイル 17
図 3-7　強制マージ形式のマニフェストファイル 18
図 3-8　ビルド用ディレクトリの作成 19
図 3-9　ビルド用ディレクトリ管理イメージ 20
図 3-10　マルチスレッドによるビルド，実行の模式図 21
図 3-11　ビルド～実行工程のステートマシン図 22
図 3-12　コンフィギュレーションスクリプト実行 24
図 3-13　AKTG 実行 26
図 3-14　make による依存性解決 28
図 3-15　make によるビルド 30
図 3-16　ターゲットボード上での実行 32
図 3-16　NIOS2 ボード上での実行 33
図 3-16　skyeye 上での実行 35

表 1-1　AKTSP テストプロセスにおける各ロール 2
表 1-2　AKTSP 配布セットの内容 4
表 2-1　AKTSP の各ディレクトリの役割 5
表 2-2　AKTB ディレクトリ内の構成 6
表 2-3　tools ディレクトリ内の構成 7
表 2-4　library ディレクトリ内の構成 7
表 2-5　tesrys ディレクトリ内の構成 8
表 2-6　yamls ディレクトリ内の構成 8

表 2-7　AKTB の実行環境 9
表 2-7　AKTB の実行環境(NIOS2 ボード) 9
表 2-7　AKTB の実行環境(skyeye シミュレータ) 9
表 3-1　AKTB の起動オプション 12
表 3-2　実行環境設定ファイル内の全体設定項目 13
表 3-3　ビルドディレクトリの状態管理ファイル名 23
表 3-4　実行環境設定ファイル内の configure 設定項目 25
表 3-5　実行環境設定ファイル内の AKTG 設定項目 27
表 3-6　NIOS2 用ケーブルの特性 33
表 3-6　実行環境設定ファイル内の全体設定項目(NIOS 2 用設定) 34
表 3-8　実行環境設定ファイル内の全体設定項目(Skyeye 用設定) 35
表 3-8　ログレベルと対応する内容 36
表 3-9　実行環境設定ファイル内のログ出力設定項目 37
表 3-10　カーネルライブラリビルドに関する設定項目 38
表 3-11　カーネルライブラリ利用に関する設定項目 38
表 4-1　終了時のログの意味 40

1. 概要

1.1　本文書の目的

本文書は，TOPPERS/ATK2 向けテストスイート AKTSP(Automotive Kernel Test Suite Package)のビルドツール，AKTB 実行に必要な情報を記述するものである．

1.2　AKTSP の概要

AKTSP は，TOPPERS プロジェクトの成果物 TOPPERS/TTSP をベースに，ATK2 向けにカスタマイズを行ったテストスイートである．以下の機能を持つ．

・PictMaster を用いた PairWise 法によるテスト設計機能
・AKTG を用いた，コーディングレスのテストソースコード自動生成機能
・AKTB を用いたテストソースコード生成～ビルド～ターゲットボード上実行までの自動実行機能

本ドキュメントは，この中で，特に AKTB についてその利用方法を記述する．また，AKTG については，AKTB から利用する範囲について，その利用法を記述する．

1.3　AKTSP テストプロセス

AKTSP では，テスト設計，並びに環境構築の作業内容が多岐に渡る．このため，以下 3 つのロールを定義して，各ロールごとに作業内容を定義する(なお，実作業上それぞれのロールを同一人物が担当する事に関し，作業上問題は無い)．

表 1-1　AKTSP テストプロセスにおける各ロール

ロール名	説明
テスト設計者	ATK2 の各機能についてテストケースの設計，実装を行う．具体的には以下の作業を行う． ・ PictMaster を使用したテストケースの設計 ・ PictMaster から生成したデータを AKC2Y ツールに入力する事による，AKTB で使用する TESRY 形式のファイルの作成
テスト環境構築者	ATK2 のターゲット環境について，AKTSP 実行に必要な環境を整備する． ・ ログ出力やテスト結果判定に用いられる AKTSP ライブラリのターゲット依存部コードの実装 ・ ターゲット用モジュールをターゲットボード上にロード，実行し，その結果をログとして取得するためのモジュールの実装 ・ 各テストケース実装のためのテスト実行環境設定ファイルの実装
テスト実施者	AKTSP を実行し，結果を確認する ・ AKTSP を実行するためのボード環境セットアップ ・ ATKB の実行，及び結果確認

各ロールごとの作業範囲を示すユースケース図，及び，AKTSP の工程における各ロールの作業を示すアクティビティ図を以下に示す．

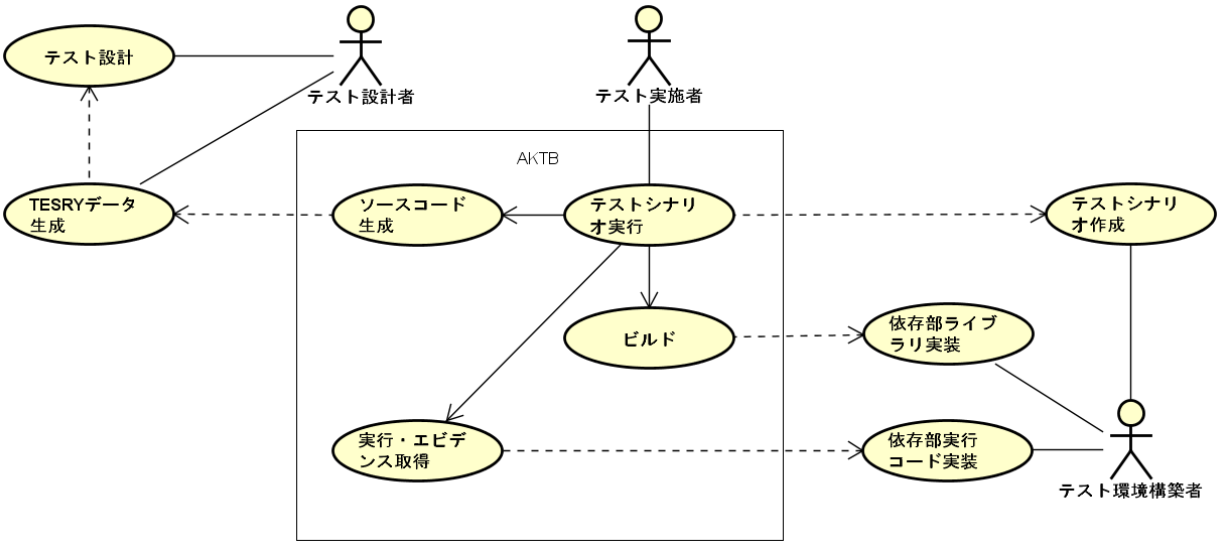


図 1-1　AKTSP のユースケース

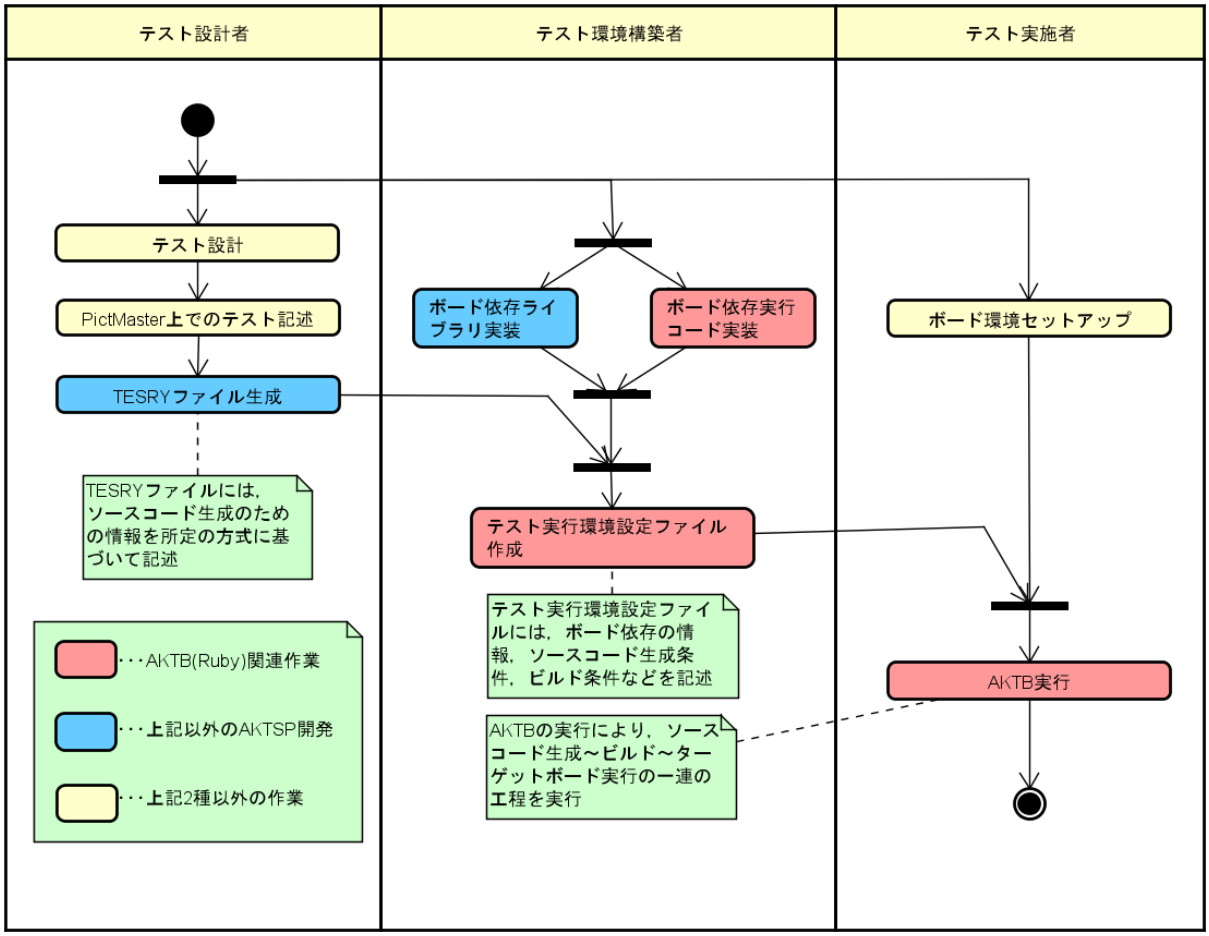


図 1-2　AKTSP テストアクティビティ

上記の図では，AKTSP の中で AKTB に直接関連するアクティビティを赤色，AKTB に直接関連しないものの，AKTB のビルドに必要な成果物を作成するアクティビティを青色，その他のアクティビティを黄色で示す.

本ドキュメントでは，主にアクティビティ「AKTB 実行」の詳細について記述する.

1.4　AKTSP 配布セットの構成

AKTSP 配布セットは，AKTB に加えて，以下のファイル群から構成される.

表 1-2　AKTSP 配布セットの内容

内容	担当ロール	内容
AKTB 一式	-	AKTB に関する Ruby スクリプトコード
TESRY ファイル	テスト設計者	ATK2-SC1, ATK2-SC1-MC, ATK2-SC3, ATK2-SC3-MC に関する全般的なテストケースを網羅.
ボード依存ライブラリ	テスト環境構築者	ハードウェア 2 種類(Skyeye(ATK2-SC1 のみ対応), Nios II)に関するライブラリソースコード
ボード依存実行コード	テスト環境構築者	ハードウェア 2 種類(Skyeye(ATK2-SC1 のみ対応), Nios II)に関する実行(ロード～ログ取得)コード
テスト実行環境設定ファイル	テスト環境構築者	ハードウェア 2 種類(Skyeye ATK2-SC1 のみ対応), NiosII)に関する上記 TESRY シート一式のテスト用設定ファイル一式

AKTSP 配布セットを使用することで，Skyeye(ATK2-SC1 のみ対応), Nios II に関する AKTSP のテストを行うことが出来るが，他のハードウェア上で実行する場合には，AKTSP のテストプロセスに従って不足分のファイルを作成する必要がある.

2. AKTB 構成

2.1　ファイル構成

2.1.1　全体構成

AKTSP を構成するファイルの全体構成を以下に示す.
以下の「手動で作成」と記述されているディレクトリは実行前に手動で作成する必要があるディレクトリである.

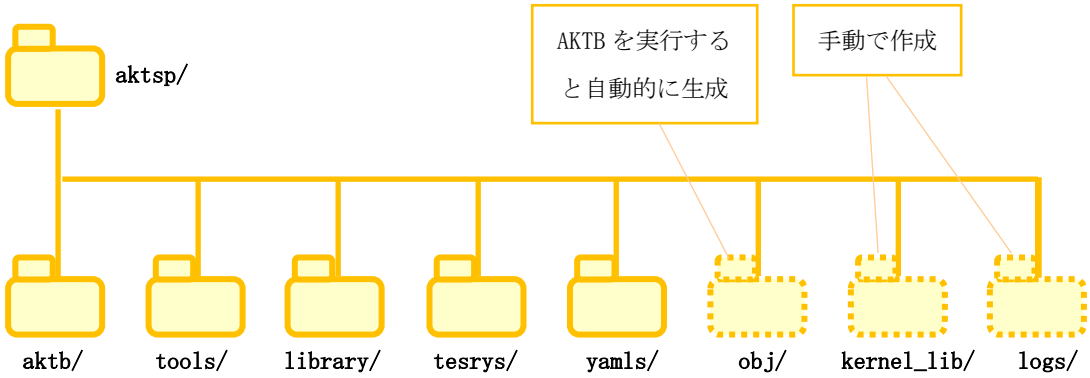


図 2-1　AKTSP 全体構成

各ディレクトリの役割を以下に示す．以降の章にて詳細を記述する.

表 2-1　AKTSP の各ディレクトリの役割

ディレクトリ名	説明
aktb/	AKTB 本体
tools/	ツール
library/	ライブラリ
tesrys/	TESRY ファイル
yamls/	テスト実行環境設定ファイル
obj/	ビルド用ディレクトリ
kernel_lib/	カーネルライブラリ用ディレクトリ
logs/	AKTB 実行ログ出力用ディレクトリ

2.1.2　AKTB 構成

aktb/ 以下の主なファイルは以下の通り.
AKTB を構成する Ruby スクリプトはこのディレクトリに格納される.

表 2-2　AKTB ディレクトリ内の構成

ディレクトリ名	説明
aktb.rb	AKTB(Ruby 版)のメインロジック
aktb.yaml	設定ファイルを指定しない場合に暗黙的に使用されるデフォルト設定ファイル
aktb_builder.rb	AKTB における一連のビルド処理を記述
aktb_log.rb	AKTB におけるログ出力ライブラリを提供
aktg_manager.rb	AKTG を使用するためのライブラリを提供
cfg_manager.rb	コンフィギュレーションスクリプトを使用するためのライブラリを提供
downloader_nios2_mc_blaster.rb	マルチコア用実行クラス 2 (NIOS 2 Blaster ケーブル)
downloader_nios2_mc_uart.rb	マルチコア用実行クラス 2 (NIOS 2 UART ケーブル)
executer_nios2_blaster.rb	非マルチコア用実行クラス（NIOS 2 Blaster ケーブル)
executer_nios2_mc.rb	マルチコア用実行クラス 1
executer_nios2_uart.rb	非マルチコア用実行クラス（NIOS2 UART ケーブル)
executer_skyeye.rb	Skyeye 用実行クラス
make_manager.rb	Make を使用するためのライブラリを提供
mnf_generator.rb	マニフェスト生成用のロジック
mp_tesry_expander.rb	メモリ拡張機能用 TESRY ファイル拡張ロジック

2.1.3　AKTSP 用ツール

tools/ の内包するディレクトリは以下の通り.

AKTSP を構成する各種ツールはこのディレクトリに格納される.

表 2-3　tools ディレクトリ内の構成

ディレクトリ名	説明
akc2y/	AKC2Y. TESRY ファイル生成機能
aktg/	AKTG. テストコード生成機能
aktm/	AKTM. カバレッジ測定ツール
coverage/	カバレッジ測定補助ライブラリ
PictMaster/	PictMaster

2.1.4　AKTSP 用ライブラリ

library/ の内包するディレクトリは以下の通り.

ハードウェア依存部や AKTSP コードを使用するためのライブラリはこのディレクトリに格納される.

表 2-4　library ディレクトリ内の構成

ディレクトリ名	説明
arch/	CPU 依存ライブラリ部を格納
check_library/	チェックライブラリ
common/	AKTSP 共通ライブラリを格納
implement/	AKTSP ターゲット依存ライブラリ実装用ライブラリを格納
target/	ターゲット依存ライブラリ部を格納

2.1.5　TESRY ファイル

tesrys/ の内包するディレクトリは以下の通り
各種 TESRY ファイルはこのディレクトリに格納される．

表 2-5　tesrys ディレクトリ内の構成

ディレクトリ名	説明
syssrv_test/	システムサービステスト
modistarc/	MODISTARC システムサービステスト
atypical_syssrv_test/	非定型型システムサービステスト
error_test/	エラーテスト
protection_test/	保護違反時処理テスト
ioc_test/	IOC 処理テスト
icisr_test/	ICISR 処理テスト

2.1.6　実行環境設定ファイル

yamls/ の内包するディレクトリは以下の通り
実行環境設定ファイルは，スケーラビリティクラス毎に分類され，このディレクトリに格納される．

表 2-6　yamls ディレクトリ内の構成

ディレクトリ名	説明
sc1/	ATK2-SC1 のテスト実行環境 YAML ファイル一式を格納
sc1-mc/	ATK2-SC1-MC のテスト実行環境 YAML ファイル一式を格納
sc3/	ATK2-SC3 のテスト実行環境 YAML ファイル一式を格納
sc3-mc/	ATK2-SC3-MC のテスト実行環境 YAML ファイル一式を格納

2.2　環境構成

本ツールは，以下の環境で利用する事を前提とする．

表 2-7　AKTB の実行環境

環境種別	プロダクト	役割
OS	Windows 7 SP1	OS
Windows 向け UNIX 環境	Cygwin 64bit 版 1.7.32	ツール起動，コンソール出力
Perl 実行環境	Perl 5.14.2	configure の実行
Ruby 実行環境	ruby 2.0.0p576	AKTB, AKTG の実行
コンフィギュレーションスクリプト	configure	Makefile の生成
テストコード生成ツール	AKTG	テスト用ソースコードの生成
make	make 4.0	生成したソースコードのビルド

また，本マニュアルでは，実行環境として，NIOS2 ボード上の実行，および skyeye シミュレータ上の実行をサポートする．NIOS2 ボードを用いた場合の実行環境を以下に示す．

表 2-8　AKTB の実行環境(NIOS2 ボード)

環境種別	プロダクト	役割
FPGA プラットフォーム	Quartus II	コンフィギュレーションの実施
NIOS2 実行ツール	Nios2-terminal	モジュールの NIOS2 へのアップロードの実施
NIOS2 ログ DL ツール	Nios2-gdb-server	実行結果の NIOS2 からのダウンロードの実施

skyeye シミュレータを用いた場合の実行環境を以下に示す

表 2-9　AKTB の実行環境(skyeye シミュレータ)

環境種別	プロダクト	役割
シミュレータ	skyeye	ATK2 向けカスタマイズを行った ARM 環境シミュレータ

3. AKTB の基本処理

3.1　処理の流れ

AKTB を用いたテスト実行の流れを以下に示す．本アクティビティ図に記載している内容については，以降の各章にて説明する．

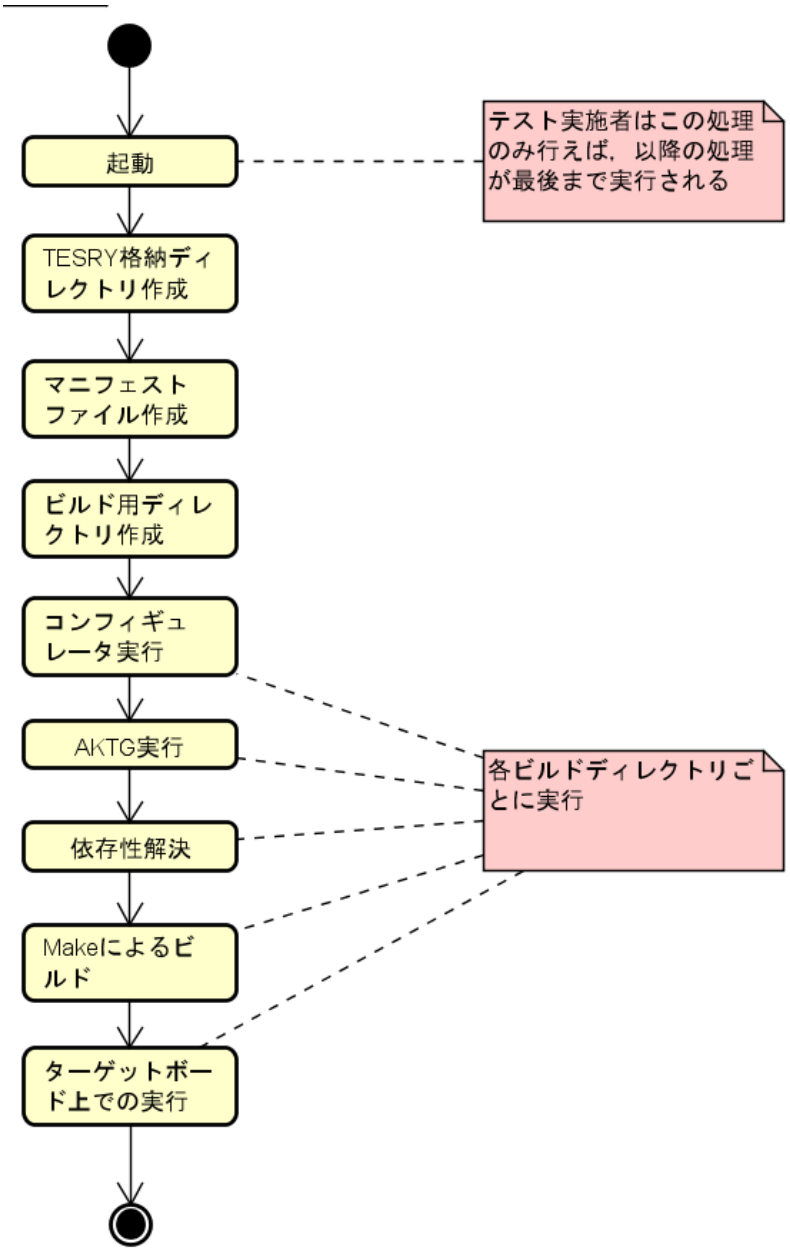


図 3-1　AKTB の処理の流れ

3.2　起動

AKTB では，起動スクリプトである aktb.rb を実行することにより処理を開始する．
AKTB では，様々な試験内容に対応するため，各テストパターンについて内容を実行環境設定ファイル(YAML 形式)に記述している．テスト実行者は，対応する実行環境設定ファイルを指定する事で，必要なテストを実行することが可能となる．

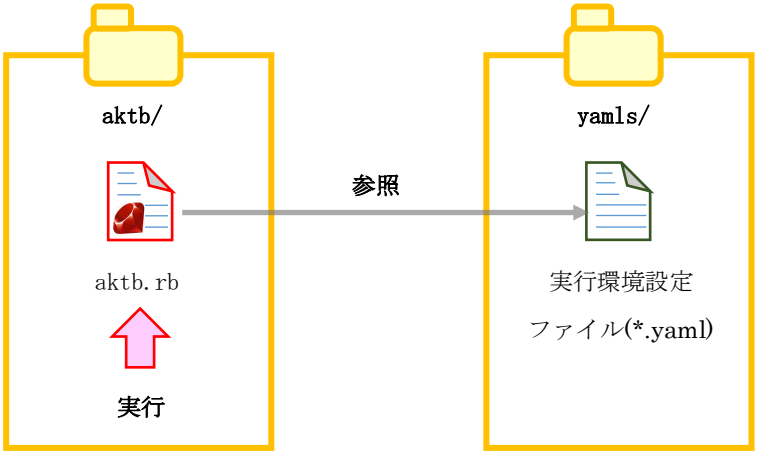


図 3-2　AKTB 起動

起動コマンドを以下に示す．以下のコマンドは aktb/ ディレクトリ内にて実行する事を前提とする．なお，コマンドに記載するパスが全て正しいファイルを参照している限り，aktb/ ディレクトリ以外の場所から実行しても問題はない．

```
$ ruby aktb.rb [–c 実行環境設定ファイルのパス] [各種オプション]
```

-c オプション(実行環境設定ファイルのパスを含む)の記述を省略した場合，カレントパスに配置されている aktb.yaml ファイルが参照される(もしこの時，カレントパスに aktb.yaml が存在しない場合はエラーとなる)．

AKTB で対応している各種オプションを以下に示す.

表 3-1　AKTB の起動オプション

ディレクトリ名	説明
[-c 実行環境設定ファイルのパス]	AKTB で使用する実行環境設定ファイルを指定する．このファイルを指定しない場合，カレントパスに存在する aktb.yaml が指定される.
[--conf 実行環境設定ファイルのパス]	同上．もし，-c と –conf の 2 つをオプションとして指定した場合は--conf に指定した内容が優先される.
--no-createdir	ディレクトリ作成モード OFF オプション. TESRY 格納ディレクトリ～ビルド用ディレクトリの工程をスキップする.
--createdir	ディレクトリ作成モード ON オプション．TESRY 格納ディレクトリ～ビルド用ディレクトリの工程を実行する．なお，ディレクトリ作成モードは ON がデフォルトである.
--no-build	ビルドモード OFF オプション．コンフィギュレーションスクリプト実行～Make によるビルドの工程をスキップする.
--build	ビルドモード ON オプション．コンフィギュレーションスクリプト実行～Make によるビルドの工程を実行する．なお，ビルドモードは ON がデフォルトである.
--no-exec	実行モード OFF オプション．ターゲットボード上での実行工程をスキップする.
--exec	実行モード ON オプション．ターゲットボード上での実行工程を実行する．なお，実行モードは ON がデフォルトである.
--build_libkernel	カーネルライブラリビルドモード ON オプション．カーネルライブラリをビルドするための処理を行う．処理の性質上，強制的にディレクトリ作成モードとビルドモードを ON に，実行モードを OFF に設定する.

また，AKTB の実行工程全体に関して，設定可能な項目は以下の通り．

表 3-2　実行環境設定ファイル内の全体設定項目

項目	書式	説明
executer_module	文字列	ハードウェア実行用モジュールのファイル名を指定．このファイルを切り替えることにより，ハードウェア，及びマルチコア/シングルコアの変更に対応可能.
divide_num	数値	AUTO_CODE 形式でビルドディレクトリを作成する場合の分割数.
use_auto_code	true/false	マージを有効とするか否かを指定.
use_force_merge	true/false	非マージ時に強制マージを有効とするか否かを指定.
max_threads	数値	マルチスレッドでビルド処理を行う際の最大同時実行数
scalability_class	文字列	スケーラビリティクラス
core_num	数値	実行環境のコア数
obj_root_path	文字列	/obj/XXXX といった形で記述．このディレクトリ以下にビルドディレクトリが作成される.
obj_tesry_path	文字列	このディレクトリ以下に TESRY ファイルがコピーされる.
cmy_path	文字列	ワイルドカードで cmy ファイルを指定する．ここで指定された CMY ファイルが obj_tesry_path 以下に展開される.
auto_dir_ladder	文字列	obj_root_path 直下のビルドディレクトリから AKTSP 直下のディレクトリまでの相対パスを指定.

3.3　TESRY 格納ディレクトリ作成

テスト用モジュールのビルド，実行を行うための作業ディレクトリ obj/ を作成し，以降はこのディレクトリ内で処理を行う．

obj/ ディレクトリ内に tesry/ ディレクトリを作成し，ここに tesrys/ ディレクトリ下にある TESRY ファイルの内容を展開する．TESRY ファイルは，内部に複数のテストケースを保持している．TESRY ファイル単位でフォルダを作成し，内包されている個々のテストケースを YAML ファイル形式で展開する．

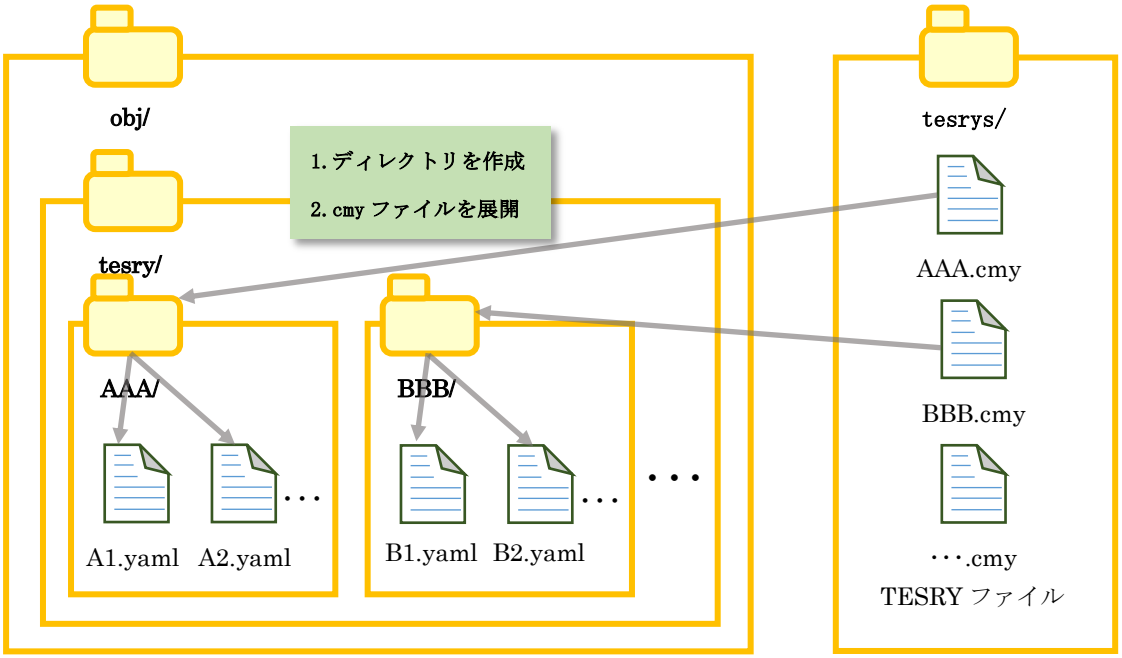


図 3-3　TESRY ディレクトリの作成

コピー元の TESRY ファイルのパスは，実行環境設定ファイルの設定項目 cmy_path にワイルドカード形式で記述されているものを用いる．

コピー先の TESRY ディレクトリのパスは，実行環境設定ファイルの設定項目 obj_tesry_path に aktsp/ ディレクトリからの相対パスで記述されているものを用いる．

3.4　マニフェストファイル作成

展開後の全ての YAML ファイルのパスを，ビルド時に参照できるように，マニフェストファイルに出力する．そして，以降の工程でビルドを行う際，ここで作成したマニフェストファイル単位でビルドディレクトリの作成，及び，ソースコードの生成，ビルドを行う．

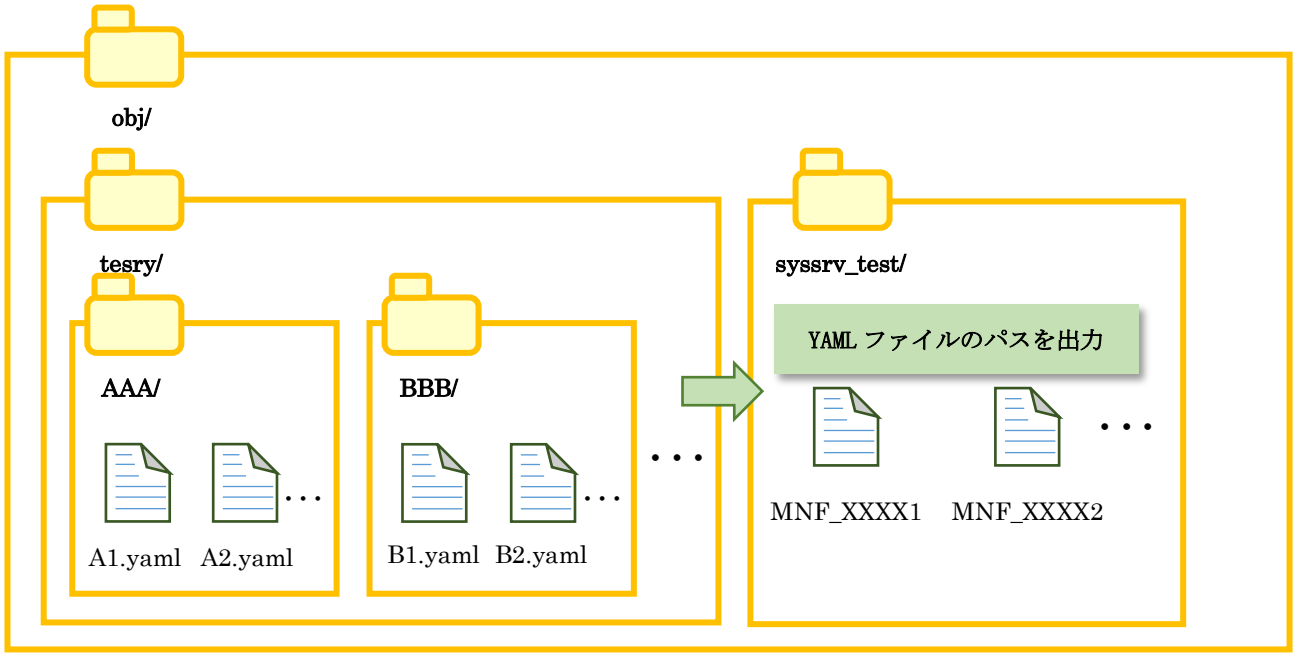


図 3-4　マニフェストファイル作成

AKTB におけるマニフェストファイルは，1 行に 1 件 YAML ファイルのパスを記述する形式を取る．例を以下に示す．YAML ファイルのパスは絶対パス形式で指定する．

```
/cygdrive/c/aktsp/obj/tesry/modistarc/ActivateTask_modistarc/ActivateTask_modistarc_0001.yaml
/cygdrive/c/aktsp/obj/tesry/modistarc/ActivateTask_modistarc/ActivateTask_modistarc_0002.yaml
/cygdrive/c/aktsp/obj/tesry/modistarc/ActivateTask_modistarc/ActivateTask_modistarc_0003.yaml
  ・・・・・
```

マニフェストファイルは，後述するマージポリシーにしたがって作成されるビルドディレクトリの名前にプレフィクス「MNF_」をつけて命名する．

通常，単一の YAML ファイルが単一の C ソースコードに対応している．しかし，テスト区分によっては，多いものでは数万単位の YAML ファイルを使用するため，そのままビルドを行うと，数万回のビルド，実行処理が発生することになる．

この回数を削減するため，モジュール内で共存可能なテストケースについては 1 つのテストケースに統合し，単一のビルドディレクトリ内で，まとめてビルド，実行作業を行う機能を用意している．これを AKTB では「マージ」と呼ぶ．
マージに関するポリシーは 3 種類存在する．以下各マージ形式の特徴について記述する．

3.4.1　AUTO_CODE(通常マージ)形式

対象の YAML ファイルを単純に指定した数に分けてグルーピングする．

例えば，YAML ファイルが 1000，分割数が 10 の場合には，YAML ファイル 100 個ずつのグループを 10 個作成する．この場合はマニフェストファイル，ビルドディレクトリはそれぞれ 10 個となる．

テスト内容によりマージを行えない TESRY ファイルも存在する(notmerge_tesry/ 以下に格納されている TESRY ファイルが該当する)．これらのテストケースは，後述の「非マージ」，もしくは「強制マージ」を用いてテストする必要がある．

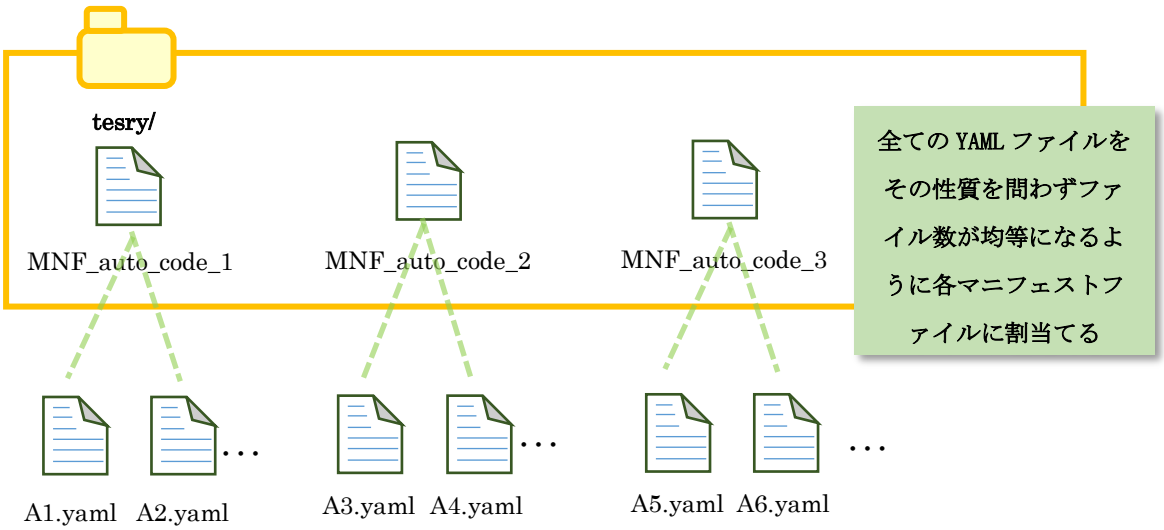


図 3-5　AUTO_CODE(通常マージ)形式のマニフェストファイル

3.4.2　非マージ形式

マージ出来ない場合のテストケースのビルド・実行形式で，YAML ファイル 1 つに 1 つのビルドディレクトリを作成する．

例えば，YAML ファイルが 1000 の場合には，YAML ファイル 1 個のグループを 1000 個作成する．この場合は，マニフェストファイル，ビルドディレクトリはそれぞれ 1000 個となる．

非常に時間がかかる形式(上記の例では 1000 回のビルド，実行処理が発生する)のため，特別な理由が無い限り，通常は，後者の強制マージを行うことで対処する．

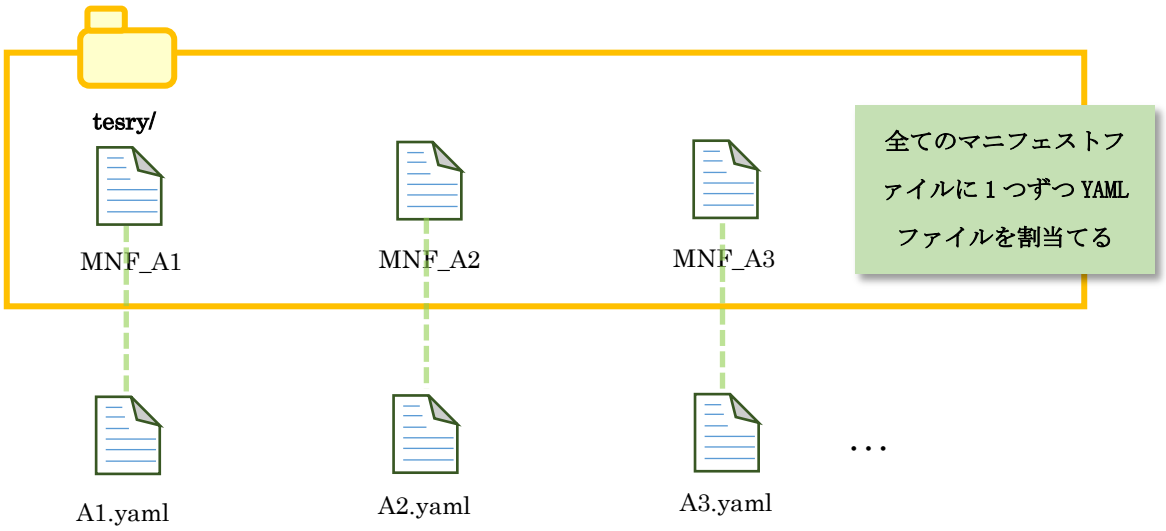


図 3-6　非マージ形式のマニフェストファイル

3.4.3　強制マージ形式

通常単一のテストソースコードに統合できないテストケースについて，各処理終了後に命令を処理開始時点にジャンプする仕組みを導入することにより，単一のソースコードに統合してのビルドが可能にになる．この仕組みを「強制マージ」と呼ぶ．

この場合は，分割前の TESRY ファイルの数だけマニフェストファイル，ビルドディレクトリが作成される．

また，強制マージ形式の場合，AKTG によるソースコード生成時にオプションを付与し，複数のソースコードを強制的に単一のソースコードにマージする必要がある．これについては後述する．

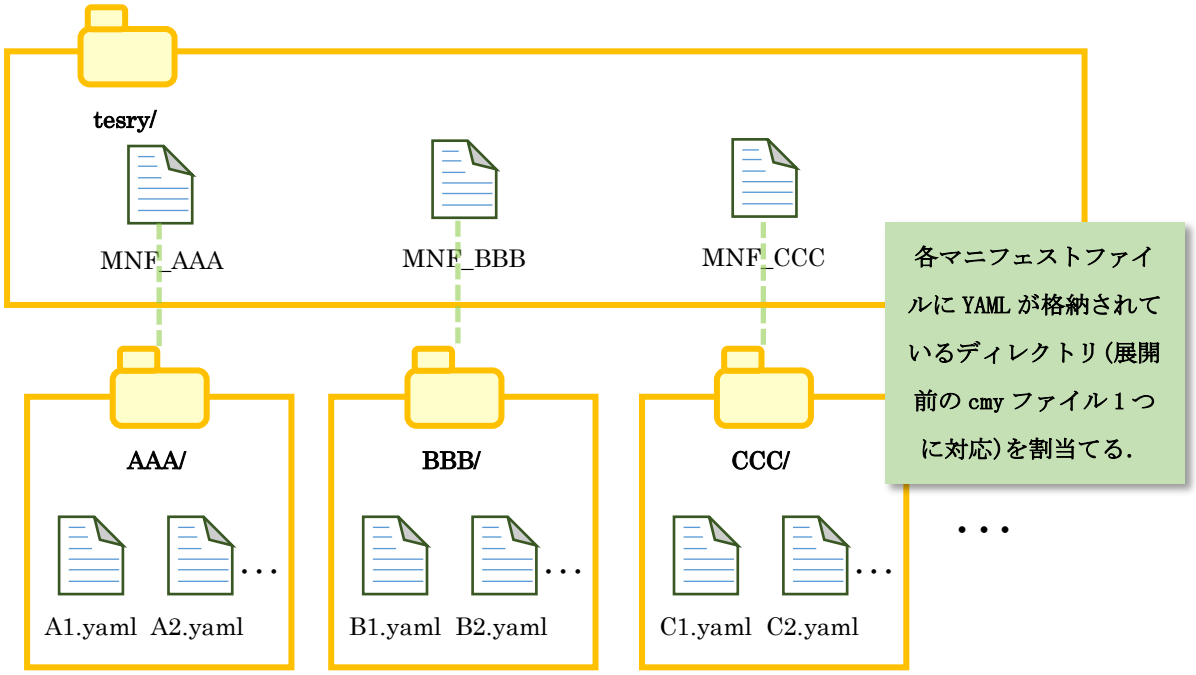


図 3-7　強制マージ形式のマニフェストファイル

3.5　ビルド用ディレクトリ作成

マニフェストファイル 1 つごとに，対応するビルドディレクトリを作成する．ディレクトリ作成後，マニフェストファイルを対応するビルドディレクトリに移動する．

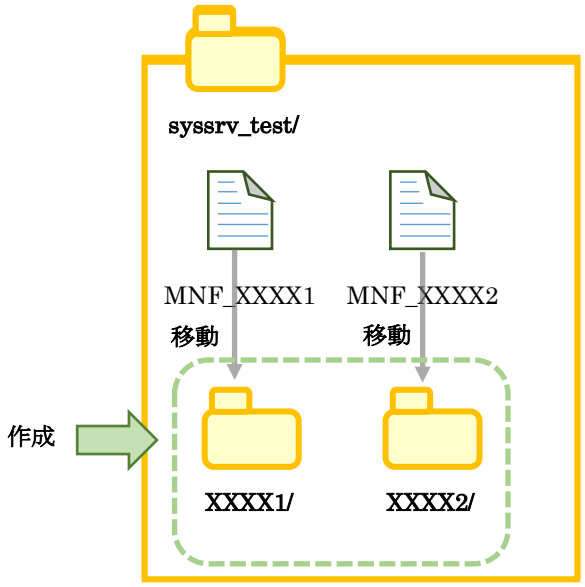


図 3-8　ビルド用ディレクトリの作成

ディレクトリ名は，マニフェストファイルのファイル名からプレフィクス「MNF_」を除いたものとする．例を以下に示す．

```
MNF_auto_code_1 ⇒ auto_code_1

MNF_osap_termination_restart_NM_a_1 ⇒ osap_termination_restart_NM_a_1
```

3.6　ビルドディレクトリ管理

AKTB では，作成した各ビルドディレクトリについて，それぞれコンフィギュレーションスクリプト実行～ターゲットボード上での実行までの各工程の処理を行う．そのため，現時点でどの状態まで処理を行っているかについて，ディレクトリ名を基にした状態管理ファイルを作成し，管理する．

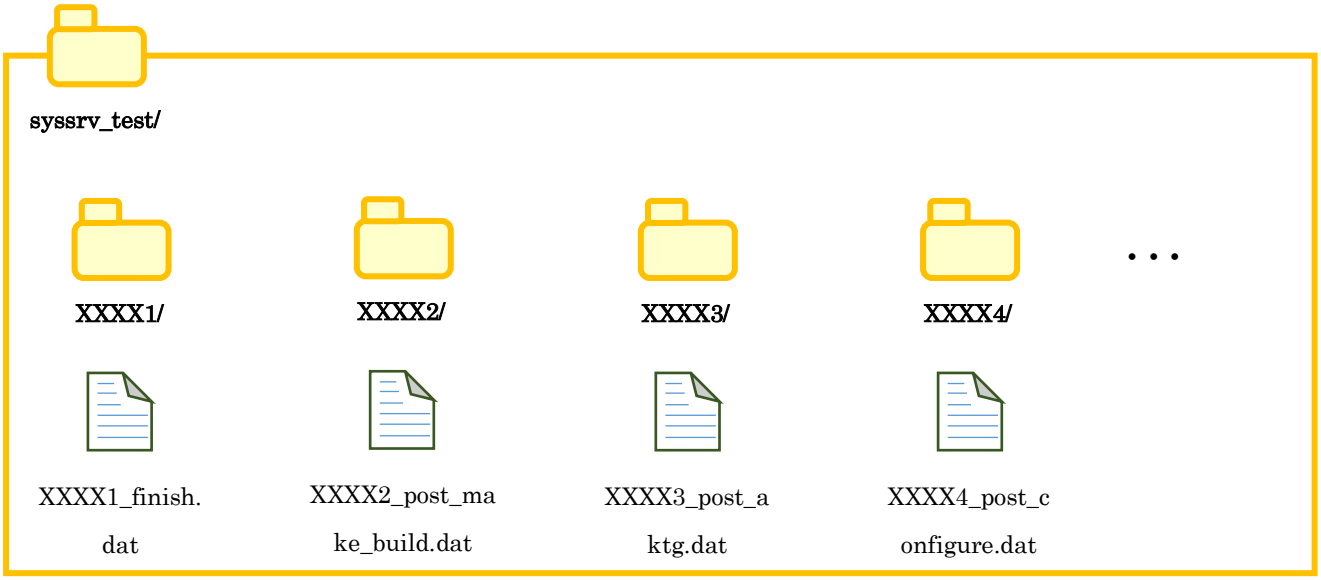


図 3-9　ビルド用ディレクトリ管理イメージ

AKTB では，マルチスレッド機構の利用により，複数のディレクトリで並行してビルド処理(コンフィギュレーションスクリプト実行～Make によるビルド)を行う．また，ターゲットボード上の実行処理についても，ビルド処理とは別のスレッドを用意して行う．

模式図を以下に示す.

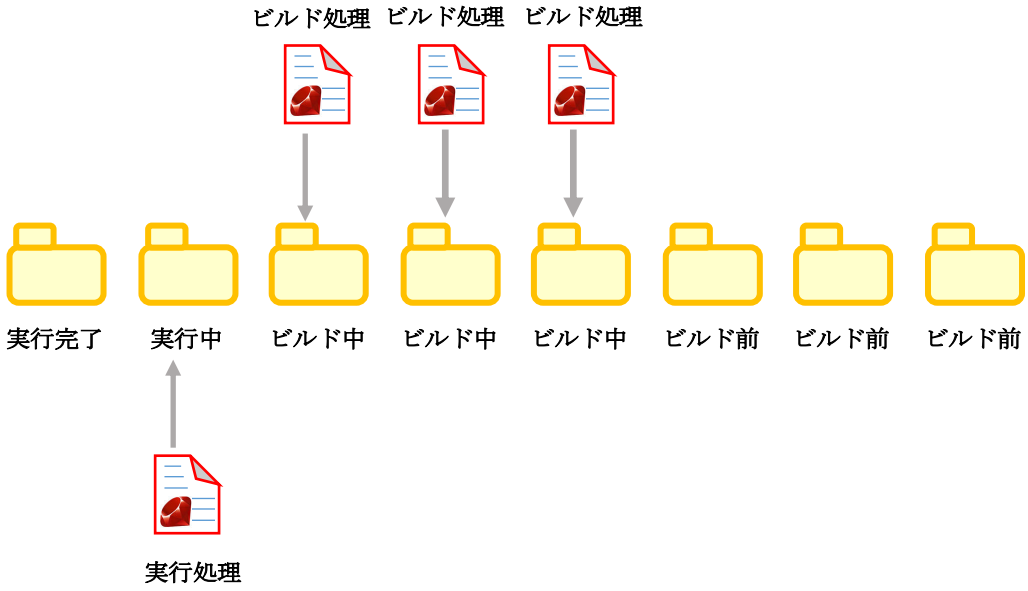


図 3-10　マルチスレッドによるビルド，実行の模式図

ビルド処理が完了したビルドディレクトリから順次ターゲットボードでの実行処理を行う．もし，実行可能な(ビルド処理が完了した)ディレクトリが存在しない場合，実行処理は新しいビルドディレクトリが生成されるまで待機する.

ビルドディレクトリのステートマシン図を以下に示す.

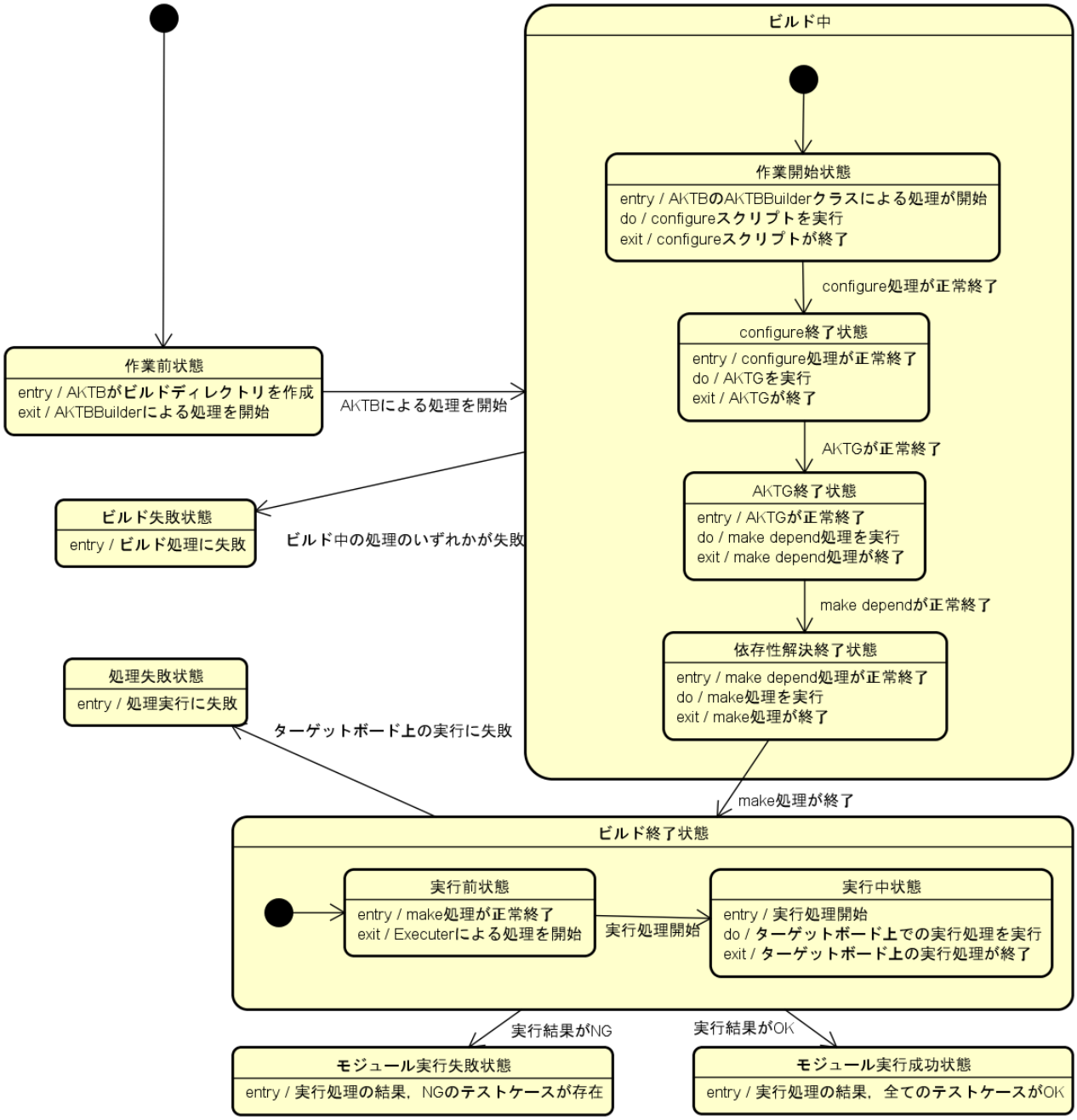


図 3-11　ビルド～実行工程のステートマシン図

AKTB は，各ビルドディレクトリが現在どの状態にあるかについて，それぞれ状態管理ファイルを生成する．状態管理ファイルはビルドディレクトリと同一のディレクトリ上に存在するファイルで，内容は存在しない(ファイルサイズは 0 バイト)．ユーザは状態管理ファイルのファイル名を確認する事により，当該ディレクトリの作業進捗確認が可能になる．ステートマシンの各状態に対応するファイル名を以下に示す．

表 3-3　ビルドディレクトリの状態管理ファイル名

状態名	ファイル名
作業前状態	(ファイルは存在しない)
作業開始状態	[ビルドディレクトリ名]_start.dat
configure 終了状態	[ビルドディレクトリ名]_post_configure.dat
AKTG 終了状態	[ビルドディレクトリ名]_post_aktg.dat
依存性解決終了状態	[ビルドディレクトリ名]_post_make_depend.dat
ビルド終了状態 (実行前状態，実行中状態共に)	[ビルドディレクトリ名]_post_make_build.dat
ビルド失敗状態	[ビルドディレクトリ名]_fail_build.dat
モジュール実行成功状態	[ビルドディレクトリ名]_finish.dat
モジュール実行失敗状態	[ビルドディレクトリ名]_ng.dat
処理失敗状態	[ビルドディレクトリ名]_fail.dat

3.7　コンフィギュレーションスクリプト実行

コンフィギュレーションスクリプトを実行し，ビルドディレクトリ内で Makefile を生成する．

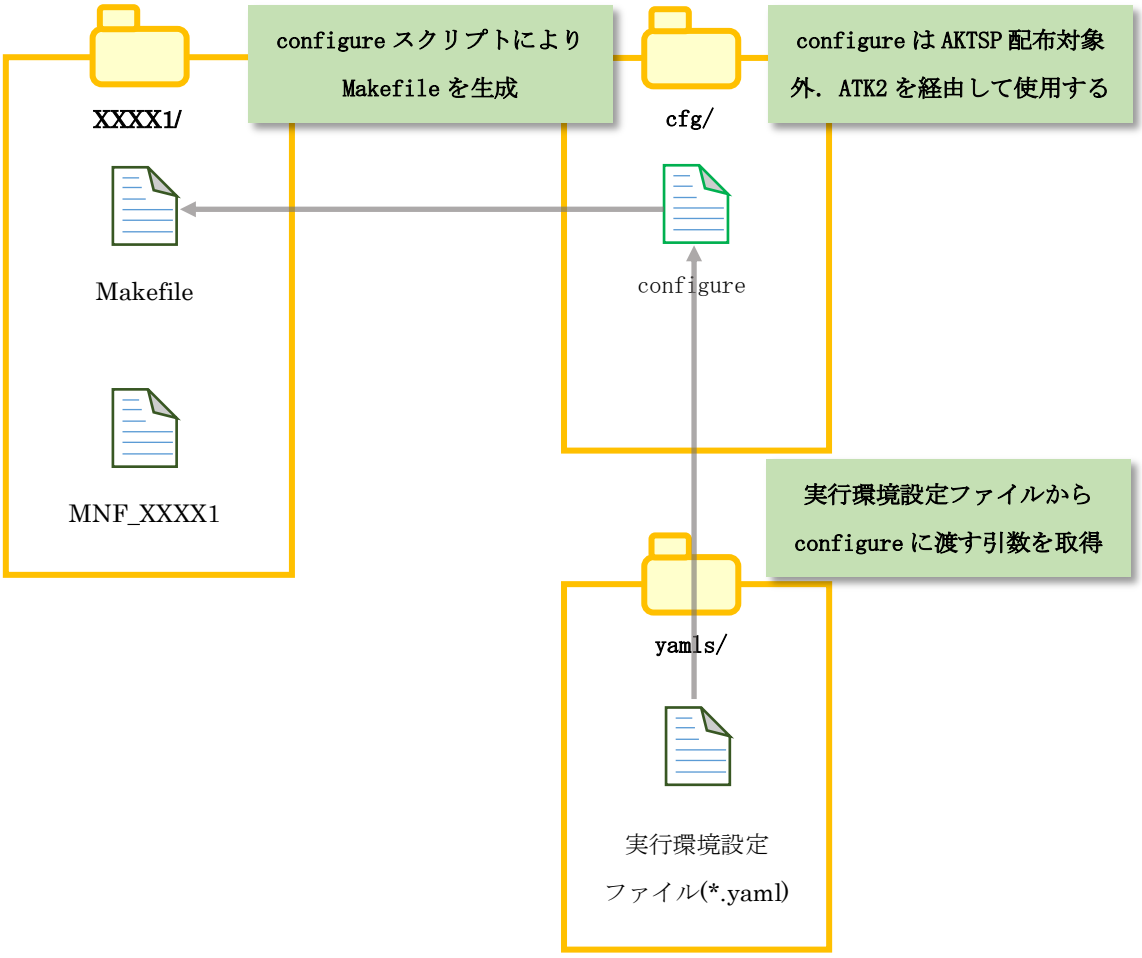


図 3-12　コンフィギュレーションスクリプト実行

configure は ATK2 の配布セットに含まれているものを使用する．
本機能では，configure に渡す変数を実行環境設定ファイルの内容をベースに生成する．

設定可能な項目は以下の通り.

表 3-4　実行環境設定ファイル内の configure 設定項目

項目	書式	説明
target_name	文字列	対象とするハードウェアターゲット
use_hwcounter	true/false	ハードウェアカウンタの使用有無
use_application_name	true/false	アプリケーション名指定の有無
application_name	文字列	アプリケーション名指定
configuration_name	リスト形式・文字列	コンフィギュレーション名
use_other_library	true/false	AKTSP ライブラリ使用の有無
other_library	リスト形式・文字列	[use_other_library=true の時のみ有効] AKTSP ライブラリのファイル名 (パス不要)
use_compile_option	true/false	共通コンパイルオプションの使用の有無
compile_option	リスト形式・文字列	[use_other_library=true の時のみ有効] 共通コンパイルオプション(COPTS に追加)
use_symbol_definition_option	true/false	シンボル定義オプションの使用の有無
symbol_definition_option	リスト形式・文字列	[use_symbol_definition_option =true の時のみ有効] シンボル定義オプション（CDEFS に追加）
include_target	リスト形式・文字列	ライブラリを参照するディレクトリを指定
use_kernel_library	true/false	カーネルライブラリの使用の有無
kernel_library	文字列	[use_kernel_library =true の時のみ有効] カーネルライブラリの場所

なお，コマンド実行結果は，ビルドディレクトリ内の result_configure.log に出力される.

3.8　AKTG 実行

AKTG を実行し，ビルドディレクトリ内でテストプログラムのソースコードを生成する．

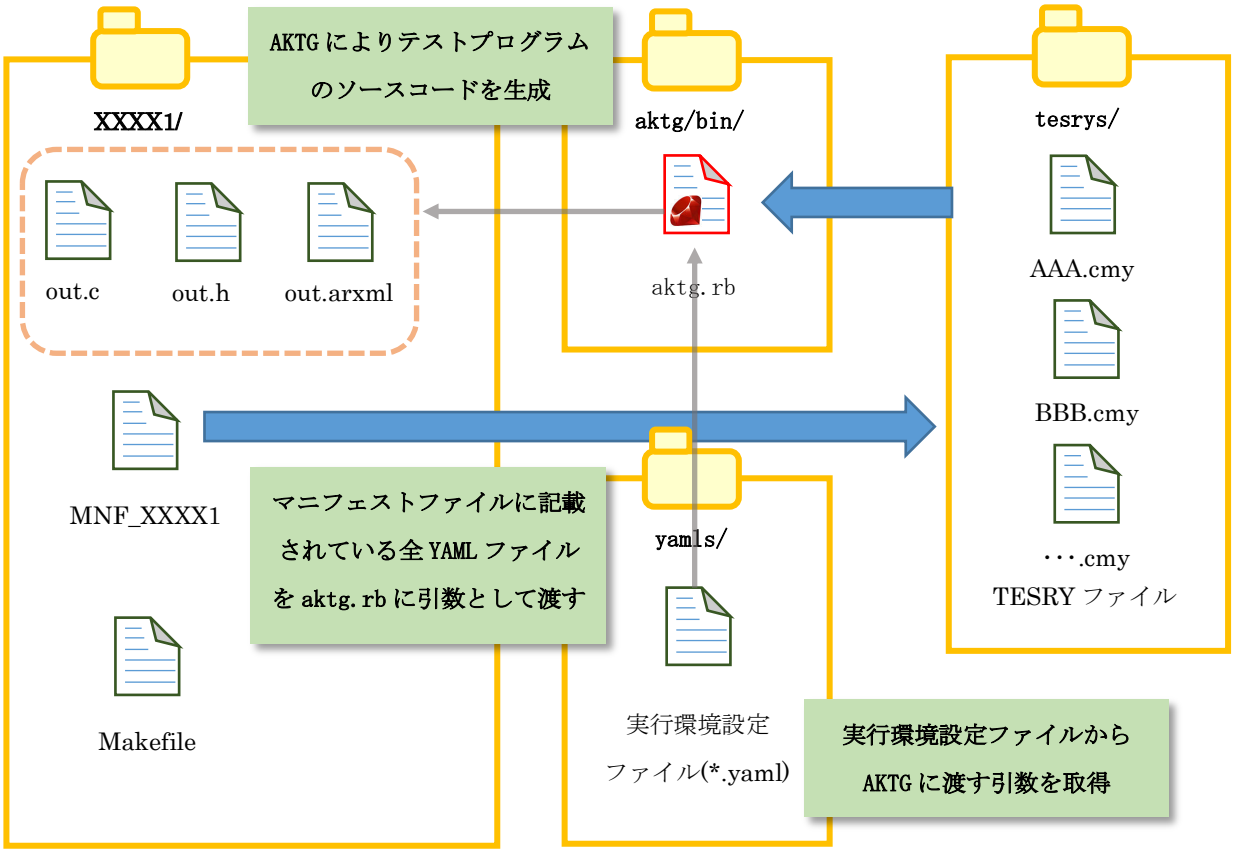


図 3-13　AKTG 実行

AKTG は AKTSP 配布セットに含まれてるものを使用する(AKTSP のルートディレクトリから見て tools/aktg/bin/aktg.rb の位置に存在する)．

本機能では，大きく分けて 2 種類の情報を生成して AKTG に渡す．1 つは AKTG の引数で，主に ATK2 のどのような機能を有効にするか，という観点から設定を行う．もう 1 つは YAML ファイルで，この YAML ファイルを基に C 言語のソースコードを生成する．YAML ファイルの場所は，以前作成したマニフェストファイルから取得する．

AKTG で設定可能な項目は以下の通り.

表 3-5　実行環境設定ファイル内の AKTG 設定項目

項目	書式	説明
tool_aktg_bin	文字列	aktg.rb のファイルが存在する場所を示すパス．AKTSP のルートディレクトリからの相対パスで記述.
scalability_class	文字列	AUTOSAR OS のスケーラビリティクラス．SC1, SC2, SC3, SC4 の 4 種類を想定している.
core_num	数字	動作する CPU のコアの数
use_multi_core	true/false	マルチコア設定を有効とするか否か. core_num の数とは独立に設定する.
use_force_merge	true/false	強制マージを有効とするか否か．この設定を ON にする場合，AKTG には引数として「-f」が渡される.
use_cpptest	true/false	C++Test 向けソース出力を有効とするか否か．この設定を ON にする場合，AKTG には引数として「-h」が渡される.

また，AKTG は多岐に渡る引数の入力を可能としている．そのため，以下の書式にて設定を行うことにより，(AKTG が許容する設定項目であれば)どの項目でも AKTG に引数として渡すことを可能とする.

```
aktg_[設定項目] : (設定内容として渡す任意の文字列)
```

以下に例を示す.

```
[実行環境設定ファイル]
aktg_stack_monitoring: false
[AKTGに渡される引数]
--stack_monitoring false
```

なお，コマンド実行結果は，ビルドディレクトリ内の result_aktg.log に出力される.

3.9　make による依存性解決

make コマンド(depend ターゲット)を実行し，OS や外部ツールへの依存性情報を Makefile.depend に出力する．また，ビルドに必要となるソースの一部をあわせて生成する．

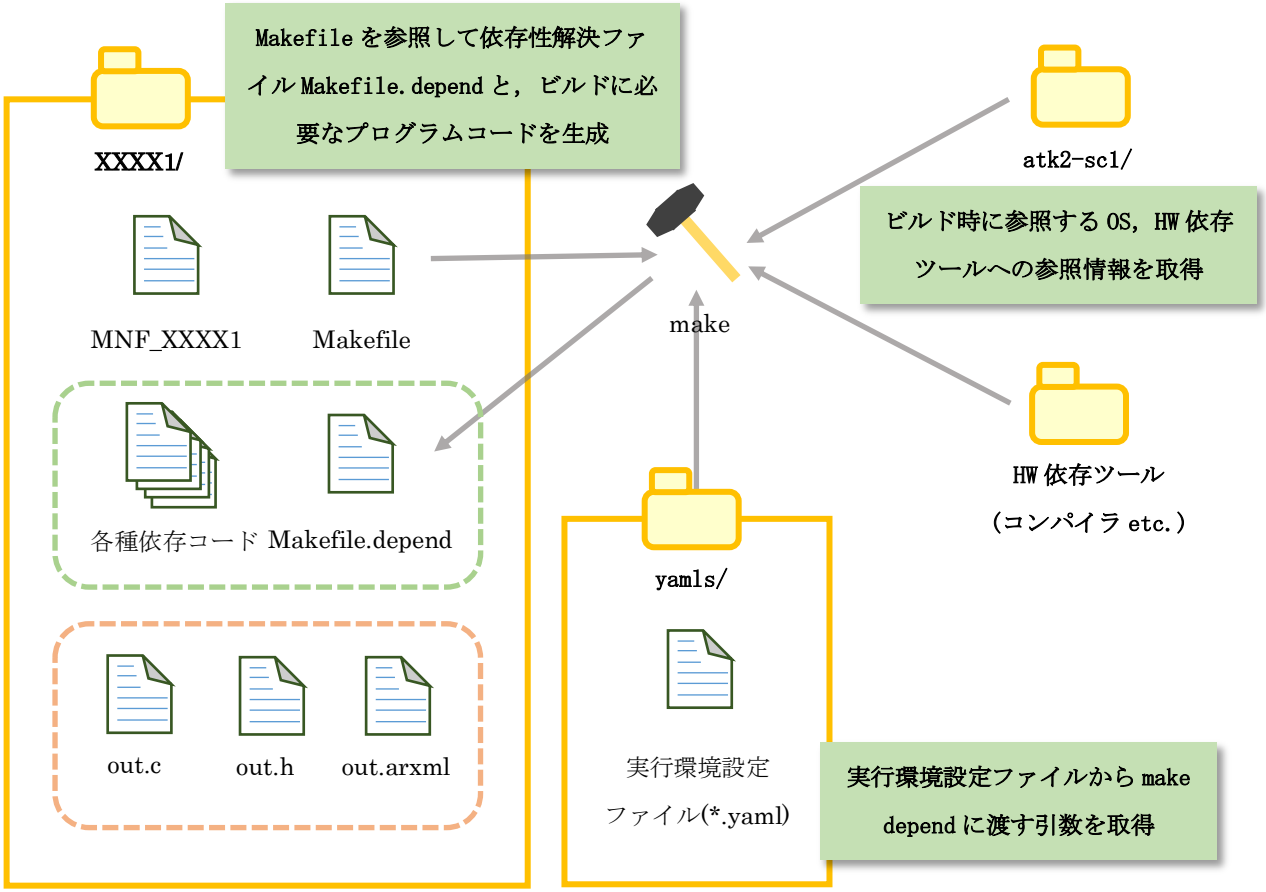


図 3-14　make による依存性解決

make は cygwin 付属のものを使用する．
本機能では，2 つの情報を make に渡す．1 つは depend ターゲットを表す引数．もう 1 つは実行環境設定ファイルに定義されている make コマンド向けのオプションである．

make depend 時に追加で渡すオプションは，実行環境設定ファイルに以下の書式で設定する．

```
make_[設定項目] : (設定内容として渡す任意の文字列)
```

以下に例を示す.

```
[実行環境設定ファイル]
make_SERIAL_UART: true
[本機能でmakeに渡される引数]
SERIAL_UART=true
```

なお，コマンド実行結果は，ビルドディレクトリ内の result_make_depend.log に出力される.

3.10　make によるビルド

make コマンド(デフォルトターゲット)を実行し，ビルド処理を行う．

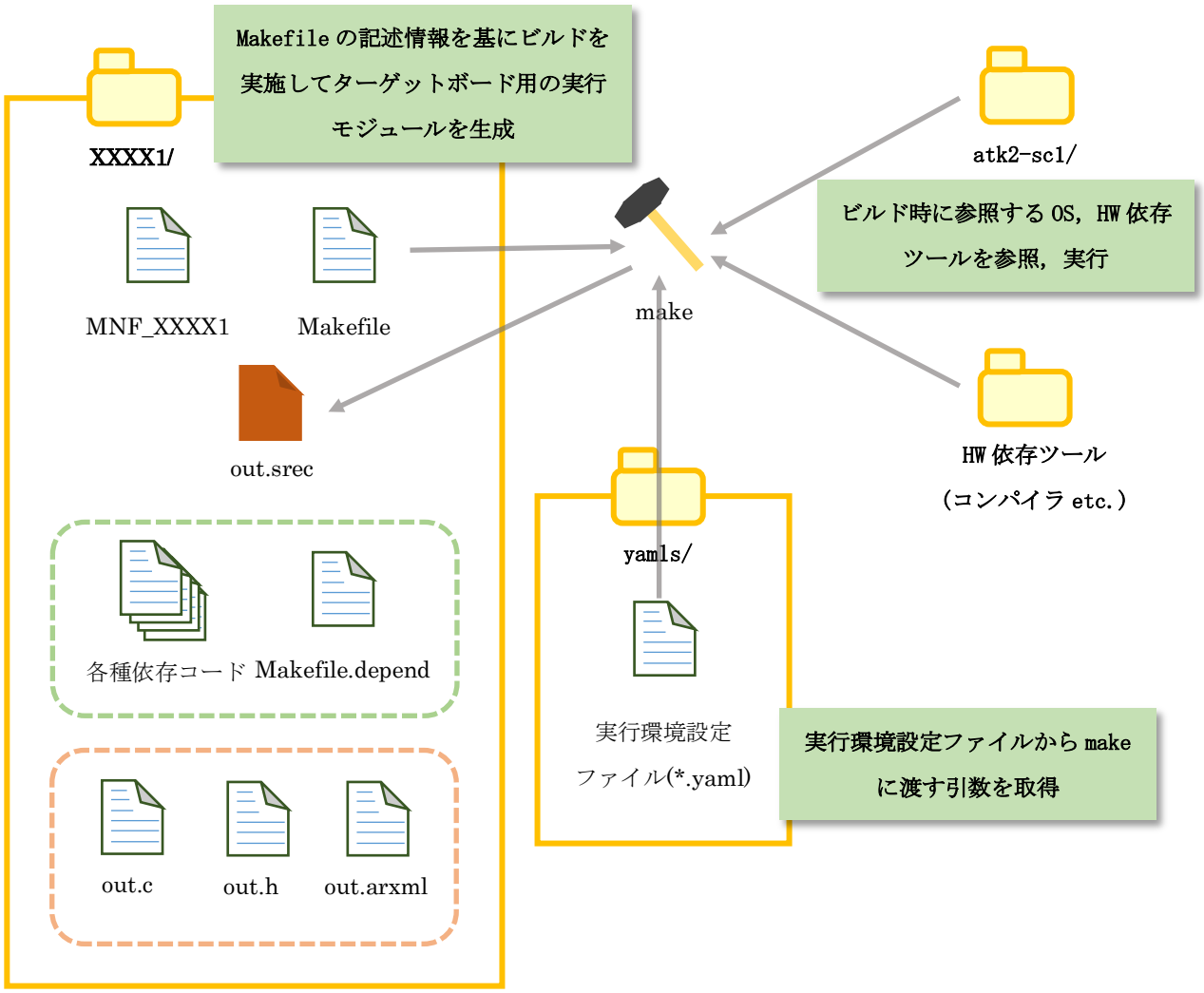


図 3-15　make によるビルド

make は cygwin 付属のものを使用する．
本機能では，2 種類の情報を make に渡す．1 つは make depend と共通で make に渡すオプション．もう 1 つは本機能のみで make に渡すオプションである．

本機能のみで make 時に追加で渡すオプションは，実行環境設定ファイルに以下の書式で設定する．

```
makebuild_[設定項目] : (設定内容として渡す任意の文字列)
```

以下に例を示す.

```
[実行環境設定ファイル]
make_SERIAL_UART: true
makebuild_USE_CPPTEST: true
[本機能でmakeに渡される引数]
SERIAL_UART=true USE_CPPTEST=true
```

なお，コマンド実行結果は，ビルドディレクトリ内の result_make_build.log に出力される.

3.11　ターゲットボード上での実行

ビルドにより作成したモジュールをターゲットボード上にロード，実行する．そして，実行結果をログとして取得し，ログ内に特定の文字列「All Check points passed」が含まれているかどうかを判定する(含まれている場合に OK と判定).

注意：強制マージ機能を利用してテストを行う場合，マージしたテストケースと同数の「All Check points passed」が出力された場合に，マージ前の全てのテストケースに合格した，とみなすことが出来る，

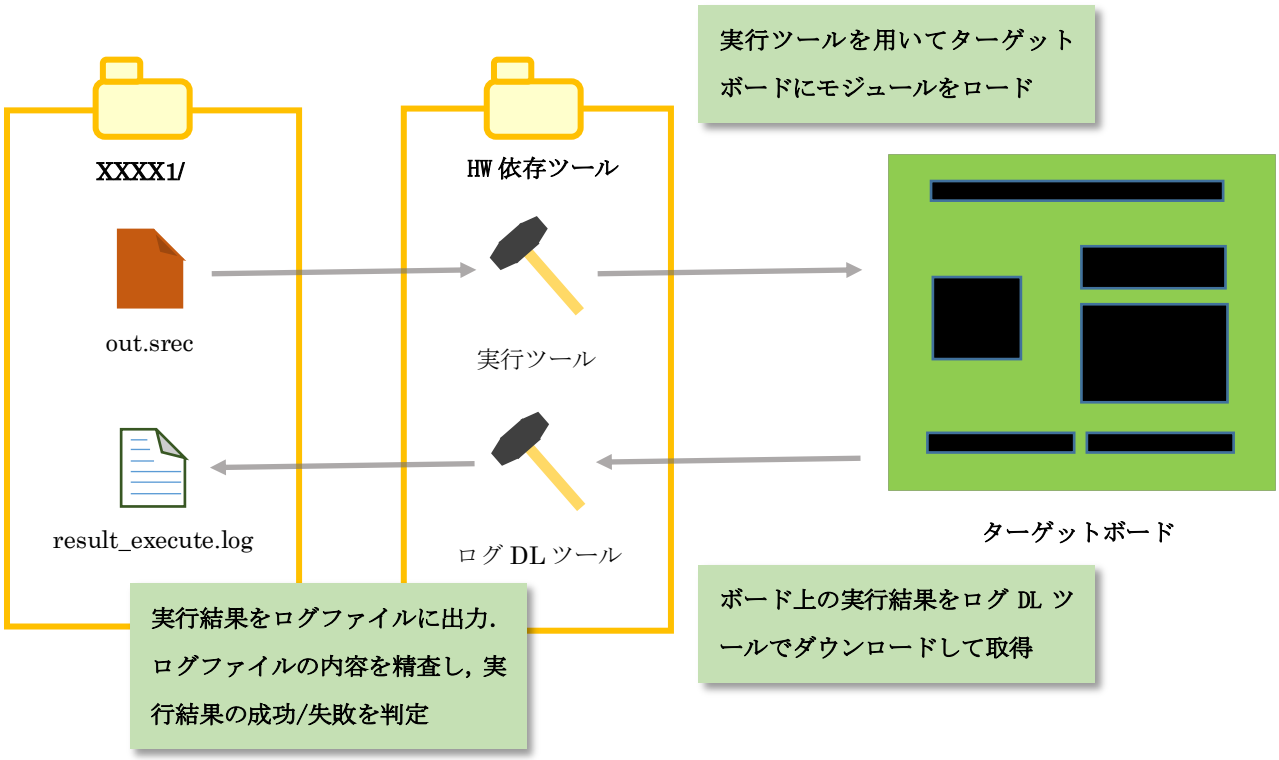


図 3-16　ターゲットボード上での実行

上記の図では，「実行ツール」と「ログ DL ツール」を個別のツールとして表記しているが，ハードウェアによりこれらのツールが統合されている可能性，もしくはさらに分割されている可能性も想定される．いずれの場合も，テスト環境構築者がテスト依存実行コードについて，実行，ログ DL による判定するよう実装が行えていればよい．

3.11.1 NIOS 2 ボード上での実行

ATK2 は，全てのスケーラビリティクラス(TP, MC 対応版含む)について NIOS2 ボード上での実行に対応する(AKTSP では，このうち，SC1, SC1-MC, SC3, SC3-MC をサポートする).
NIOS2 ボード上で ATK2 モジュールを実行する場合，PC と NIOS2 ボードの間でのデータのやり取りを 2 種類のケーブル(Blaster ケーブル，UART ケーブル)で行うことが出来る事に留意する必要がある.

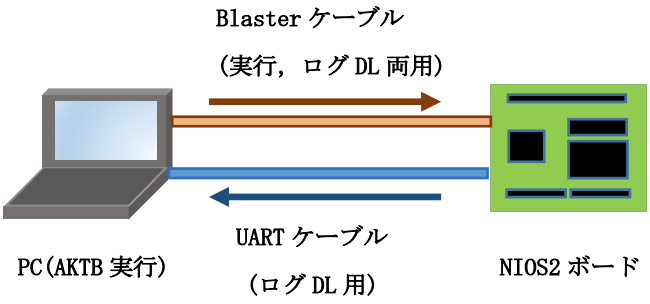


図 3-17　NIOS2 ボード上での実行

2 種類のケーブルについての主な特徴を以下に示す.

表 3-6　NIOS2 用ケーブルの特性

ケーブル種別	特性
Blaster ケーブル	・ 実行，ログ DL の両方を行うことが出来る ・ 通信速度は低速
UART ケーブル	・ ログ DL のみを行うことが出来る(UART ケーブルを使用する場合であっても，実行処理は Blaster ケーブル経由で行う必要がある) ・ 通信速度は高速

ケーブル毎にログ DL 時の処理が異なることから，ケーブルにより，Ruby モジュールを切り替えて処理する必要がある.
また，マルチコアの処理を行う場合には，マスタコアとスレーブコアそれぞれに対してモジュールの実行とログの DL を行う必要があるため，シングルコア向けのテストを行う場合と，マルチコア向けのテストを行う場合では，異なる Ruby モジュールを用いる必要がある.

ケーブルの違いや，マルチコア/シングルコアの違いによる処理の切り替えは，設定ファイルで対応する Ruby モジュールを選択することにより行う．

表 3-7　実行環境設定ファイル内の全体設定項目(NIOS 2 用設定)

項目	書式	説明
target_exec_cmd	文字列	「実行ツール」の実行コマンド
target_log_cmd	文字列	「ログ DL ツール」の実行コマンド
max_exec_time	数値	実行処理の制限時間(単位 ： 秒)
downloader_module	文字列	ログ DL 関連処理を行う Ruby モジュールの名前
executer_module	文字列	ハードウェア実行用モジュールのファイル名を指定．このファイルを切り替えることにより，ハードウェア，及びマルチコア/シングルコアの変更に対応可能

3.11.2 skyeye 上での実行

ATK2 は，プロセッサのオープンソースシミュレータである skyeye 上で実行する事が可能である(ただし SC1 限定)．この環境を利用して AKTSP によるテストを行うことが出来る(SC1 限定).



PC(AKTB 実行)

図 3-18　skyeye 上での実行

シミュレータ上の実行となるため，skyeye の場合は，ターゲット実行，DL 処理にケーブルを必要としない．また，マルチコア向け ATK2 は skyeye 上で実行する事が出来ない．そのため，skyeye によるテストを行う場合は，Ruby モジュールの切り替えは発生しない(単に skyeye 向けの Ruby モジュールを用いればよい).

表 3-8　実行環境設定ファイル内の全体設定項目(Skyeye 用設定)

項目	書式	説明
target_exec_bin	文字列	「実行ツール」(兼「ログ DL ツール」)の実行コマンド
target_exec_conf	文字列	「実行ツール」(兼「ログ DL ツール」)の設定ファイル
max_exec_time	数値	実行処理の制限時間(単位 ： 秒)
executer_module	文字列	ハードウェア実行用モジュールのファイル名を指定．このファイルを切り替えることにより，ハードウェア，及びマルチコア/シングルコアの変更に対応可能

3.12 共通処理

以下，各工程で共通して利用する処理について記述する．

3.12.1 ログ出力

実行環境設定ファイルの log_path に指定したパスに aktb_logger.log という名前でログが出力される．出力するログの例を以下に示す．

```
# Logfile created on 2015-06-17 16:26:06 +0900 by logger.rb/41954
I, [2015-06-17 16:26:06#5052]  INFO -- : <<< AKTSP START >>>
I, [2015-06-17 16:26:06#5052]  INFO -- : [Config File] aktb.yaml
I, [2015-06-17 16:26:06#5052]  INFO -- : [divide_cmy] START
I, [2015-06-17 16:27:18#5052]  INFO -- : [divide_cmy] FINISH
I, [2015-06-17 16:27:18#5052]  INFO -- : [generate_manifest] START
I, [2015-06-17 16:27:19#5052]  INFO -- : [generate_manifest] FINISH
I, [2015-06-17 16:27:19#5052]  INFO -- : [divide_manifest] START
. . .
```

本ログ機能は，Ruby の標準ライブラリ'logger'を用いて実装しているため，ログレベルもこの機能のレベルに準じる．ログレベルは 5 種類存在する．デバッグメッセージ等，実運用上参照する必要性が低いログ程低いログレベルが割当てられる一方，エラー等，実運用上参照する必要性が高いログ程，高いログレベルが割当てられる．
以下，各レベルについての説明を示す．ログレベルは，表の上に位置するほど高くなる．

表 3-9　ログレベルと対応する内容

ログレベル	書式
FATAL	プログラムをクラッシュさせるような制御不可能なエラー
ERROR	エラー
WARN	警告
INFO	一般的な情報
DEBUG	低レベルの情報

アプリケーション側の設定により，レベルに応じてファイルに出力するログをフィルタリングする事が出来る．例えば，「WARN レベル」を設定した場合，FATAL～WARN レベルに該当するログ情報のみがログファイルに出力される．なお，出力レベル設定のデフォルトは「INFO レベル」である．

表 3-10　実行環境設定ファイル内のログ出力設定項目

項目	書式	説明
log_file_path	文字列	ログファイルの出力先を示すパス．AKTSP のルートディレクトリからの相対パスで記述．本設定が無い場合，カレントディレクトリに aktb_logger.log が出力される．
log_level	文字列	ログのフィルタリングレベルを示す．「Logger::ログレベル」形式で記述する．設定しない場合は” Logger::INFO” が設定される．

3.12.2 処理中断・再開

AKTB では，ビルドディレクトリが全て作成された後であれば，起動後に処理を中断し，後で改めて再開する事が可能である．

ビルドディレクトリが全て作成されたかどうかを判断するには，ビルドディレクトリが存在するディレクトリにおいて，状態管理ファイルの拡張子である「*.dat」を含むファイルが 1 つ以上存在する事を確認すればよい．

再開の際，AKTB を起動する際に以下の起動オプションにて実行する．--no-createdir オプションを除いて実行すると，ディレクトリ作成の段階から処理が戻ってしまうため，注意が必要である．

```
$ ruby aktb.rb [–c 前回実行時と同じ実行環境設定ファイルのパス] --no-createdir
```

3.13 特殊処理

3.13.1 カーネルライブラリビルド

ATK2 ソースコードを予めコンパイル済みのライブラリとして用意し， 参照することにより AKTB の処理時間を短縮することが出来る．

カーネルライブラリビルドモードを ON にし，以下の設定項目を設定した実行環境設定ファイルを指定して aktb.rb を実行した場合に，obj/ 内にカーネルライブラリ libkernel.a が生成される．

表 3-11　カーネルライブラリビルドに関する設定項目

項目	書式	説明
target_libkernel_yml	文字列	libkernel ビルド用に作成された最小単位の TESRY ファイル

3.13.2 カーネルライブラリの使用

カーネルライブラリを指定してビルドを行う場合は，以下の設定項目を設定した実行環境設定ファイルを指定して aktb.rb を実行する事．

表 3-12　カーネルライブラリ利用に関する設定項目

項目	書式	説明
use_kernel_library	true/false	カーネルライブラリの使用の有無
kernel_library	文字列	[use_kernel_library =true の時のみ有効] カーネルライブラリの場所

3.13.3 C++Test によるカバレッジ計測

AKTSP では，C++Test のライブラリを用いて，カバレッジ計測用のデータを取得できる．
この作業は以下の手順で行う．

1. C++Test の機能を使用して，カーネルライブラリ libkernel.a を作成(詳細はカバレッジ計測手順書を参照の事)
2. 上記の条件で作成した libkernel.a，及び C++Test 用ファイルを kernel_lib/ディレクトリに配置の上，C++Test 専用のオプションを指定してテストを実行⇒実行ログファイルに C++Test 向けのカバレッジ情報(CLOG)が出力

工程 2 で，カバレッジ情報取得の為に，kernel_lib ディレクトリに配置すべきファイルは以下の通り

libcpptest.a libkernel.a （上記工程で作成） Os_Cfg.h

工程 2 で，カバレッジ情報取得のために，追加で設定する項目は以下の通り．

use_cpptest: true use_kernel_library: true kernel_library: kernel_lib (←libkernel.a 等を配置したディレクトリ) aktg_enable_cpptest: true make_USE_CPPTEST: true make_NDEBUG: true make_CPPTEST_USE_BUFFER: true make_CPPTEST_BUFFER_SIZE: 143360

4. 成功，失敗判定

AKTB はビルドディレクトリ単位で成功，失敗件数を管理する．全ビルドディレクトリにおいてターゲット実行が終了した時点で，以下の形式のログが出力される．

```
I, [2015-07-30 15:16:09#3596]  INFO -- : [target->all:1,exec:1]
I, [2015-07-30 15:16:10#3596]  INFO -- : [target] FINISH
I, [2015-07-30 15:16:10#3596]  INFO -- : Test Case : 1
I, [2015-07-30 15:16:10#3596]  INFO -- : OK : 1
I, [2015-07-30 15:16:10#3596]  INFO -- : NG : 0
I, [2015-07-30 15:16:10#3596]  INFO -- : EXEC FAILED : 0
I, [2015-07-30 15:16:10#3596]  INFO -- : BUILD FAILED : 0
I, [2015-07-30 15:16:10#3596]  INFO -- : [build_and_execute] FINISH
I, [2015-07-30 15:16:10#3596]  INFO -- : <<< AKTSP FINISH >>>
```

上記ログで出力されている Test Case からの 5 行分のログの意味は以下の通り．

表 4-1　終了時のログの意味

ログレベル	書式
Test Case	全テストケース数(ビルドディレクトリ数と同じ)
OK	「モジュール実行成功状態」(ビルド，実行に成功した，全てのテストケースをパス)に該当するテストの件数
NG	「モジュール実行失敗状態」(ビルド，実行に成功したが，パスしなかったテストケースが存在)に該当するテストの件数
EXEC FAILED	「処理失敗状態」(処理実行に失敗)に該当するテストの件数
BUILD FAILED	「ビルド失敗状態」(ビルドに失敗)に該当するテストの件数

AKTB では「Test Case 数 = OK 数」の関係が成立した場合に，テストが成功したものとみなす．NG や EXEC FAILED，BUILD FAILED が存在する場合は，表 3-3 を参考に上記に対応する状態管理ファイル名を探すことにより，問題が発生しているビルドディレクトリを見つけることが出来る．

5. 変更履歴

Version	Date	Detail	Editor	Authorizer
0.9.0	2015/08/28	新規作成	NCES	
0.9.2	2015/10/31	社内レビューの指摘を反映	NCES	